



МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

**ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И
ТЕХНОЛОГИЧЕСКОГО ОБРАЗОВАНИЯ**
Кафедра информационных технологий и электронного обучения

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного обеспечения»
форма обучения – очная

Курсовая работа

«Разработка веб-портфолио программиста с использованием Lektor CMS»

Обучающейся 4 курса
Закаблукковой Анастасии Эдуардовны

Научный руководитель:
кандидат физико-математических наук,
доцент кафедры ИТиЭО
Жуков Николай Николаевич

Санкт-Петербург
2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	2
1 Теоретические аспекты создания веб-портфолио.....	4
1.1 Назначение веб-портфолио программиста.....	5
1.2 Основные виды и структура веб-портфолио.....	5
2 Проектирование и реализация веб-портфолио с помощью Lektor CMS.....	10
2.1 Проектирование структуры, функциональных требований и дизайн веб-портфолио.....	11
2.2 Реализация основных разделов и функциональных модулей в Lektor CMS..	12
2.3 Развертывание и публикация веб-портфолио на GitHub Pages.....	15
ЗАКЛЮЧЕНИЕ.....	16
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ.....	17
ПРИЛОЖЕНИЕ А.....	19
ПРИЛОЖЕНИЕ Б.....	20
ПРИЛОЖЕНИЕ В.....	21
ПРИЛОЖЕНИЕ Г.....	22

ВВЕДЕНИЕ

Актуальность темы обусловлена стремительной цифровизацией общества и возрастающими требованиями к профессиональной самопрезентации специалистов в сфере информационных технологий. В условиях высокой конкуренции на рынке труда наличие персонального веб-портфолио становится важным инструментом демонстрации уровня профессиональной подготовки, практических навыков и реализованных проектов. В международной практике портфолио рассматривается как ключевой элемент формирования профессиональной репутации специалиста и способ повышения его конкурентоспособности. [1, 2]

Качественно разработанное веб-портфолио способствует более эффективному позиционированию программиста, позволяя систематизировать и наглядно представлять его достижения, опыт и компетенции. Таким образом, задача создания функционального и эстетически оформленного веб-портфолио является актуальной, а ее решение демонстрирует практическое применение современных веб-технологий и инструментов разработки.

Целью данной курсовой работы является разработка веб-портфолио программиста с использованием системы управления контентом Lektor CMS.

Для достижения поставленной цели в работе предполагается решение следующих **задач**:

- проанализировать современные подходы к созданию профессиональных веб-портфолио;
- изучить технические особенности и преимущества статического генератора Lektor CMS;
- спроектировать структуру и дизайн веб-портфолио;
- реализовать основные функциональные модули сайта;
- настроить хостинг и опубликовать сайт.

Объект работы: процесс разработки профессионального веб-портфолио программиста.

Предмет работы: методы и средства создания портфолио на основе статических генераторов сайтов Lektor CMS.

Структура курсовой работы включает в себя введение, две главы, заключение и список литературы, включающий 13 источников. Работа содержит 24 страниц, 5 рисунков, 1 таблицу и 4 приложения. В первой главе рассматриваются теоретические аспекты создания веб-портфолио и обоснование выбора инструментальных средств разработки, во второй - процесс проектирования, разработки и публикация веб-портфолио.

Теоретической базой исследования послужили научные публикации, учебные и методические материалы, посвященные веб-разработке и самопрезентации в профессиональной среде, а также официальная техническая документация и практические руководства по работе с системой Lektor CMS.

1 Теоретические аспекты создания веб-портфолио

1.1 Назначение веб-портфолио программиста

Веб-портфолио программиста представляет собой персональный сайт, содержащий структурированную информацию о профессиональных навыках, реализованных проектах, опыте работы, уровне образования, а также дополнительных достижениях в профессиональной сфере. По своей сути оно является цифровым отражением профессионального профиля специалиста и служит инструментом демонстрации его компетенций потенциальным работодателям, заказчикам или партнёрам.

Ключевое назначение веб-портфолио заключается в систематизированном представлении результатов деятельности программиста. В отличие от традиционного резюме, которое, как правило, носит краткий описательный характер, веб-портфолио предоставляет возможность продемонстрировать реальные продукты своей работы: веб-сайты, приложения, фрагменты кода, дизайн-решения, техническую документацию и другие цифровые материалы. Таким образом, портфолио выполняет не только источник информации, но и средством наглядной демонстрации уровня подготовки специалиста на основе конкретных примеров. [3, 4]

1.2 Основные виды и структура веб-портфолио

Веб-портфолио может иметь различные формы и назначение в зависимости от целей специалиста, его профессиональной направленности и целевой аудитории. Несмотря на индивидуальный характер каждого портфолио, в практике выделяют несколько основных видов, отличающихся по структуре, содержанию и функциональному наполнению.

Одним из наиболее распространённых является персональное портфолио, ориентированное на самопрезентацию конкретного специалиста. Оно содержит

информацию о профессиональных навыках, опыте работы, реализованных проектах, достижениях и контактных данных.

Вторым видом является проектное портфолио, в котором акцент делается на демонстрации выполненных работ. В отличие от персонального, здесь на первый план выходит не биография программиста, а его практическая деятельность. Основное содержание составляют описания проектов, скриншоты, ссылки на работающие версии приложений или репозитории исходного кода, технические характеристики и применяемые технологии.

Также выделяется образовательное портфолио, в котором основное внимание уделяется этапам обучения, проделанным учебным и исследовательским работам, курсам, сертификатам и полученным компетенциям. Этот вид портфолио характерен для студентов и начинающих специалистов, не имеющих ещё значительного коммерческого опыта, но желающих продемонстрировать уровень своей подготовки и стремление к профессиональному развитию.

По реализации портфолио делятся на статические и динамические. Статические сайты обладают высокой скоростью, безопасностью и не требуют серверной логики. Динамические используют базы данных и подходят для расширенного функционала, но менее рациональны для простой демонстрации работ.

Был проведен анализ существующих веб-портфолио, ссылки на которые представлены в приложении А. [5, 6] Этот анализ позволил выявить общие паттерны, оценить разнообразие визуальных и структурных подходов. Независимо от выбранного типа, структура веб-портфолио включает несколько обязательных разделов. [7, 8] К ним относятся:

- главная страница, формирующая первое впечатление о специалисте и его деятельности;

- раздел “О себе”, содержащий информацию об образовании, опыте и профессиональных интересах;
- раздел “Портфолио”, включающий работы и проектами с их описание;
- раздел “Контакты” с формой обратной связи и ссылками на профессиональные профили.

1.3 Современные технологии и инструменты для разработки веб-портфолио

Современные инструменты для создания веб-портфолио варьируются от ручной разработки с использованием HTML, CSS и JavaScript до применения фреймворков, систем управления контентом и статических генераторов сайтов. Выбор конкретного инструмента зависит от уровня подготовки разработчика, целей проекта и требований к функциональности и сопровождению веб-ресурса.

Ручная разработка сайта с использованием базовых веб-технологий обеспечивает полный контроль над кодом и архитектурой проекта, однако требует значительных временных затрат, особенно при необходимости регулярного обновления контента. Применение фреймворков и библиотек позволяет ускорить процесс разработки и создать современный пользовательский интерфейс, но зачастую усложняет архитектуру проекта и не всегда оправдано для информационного ресурса, каким является веб-портфолио.

Традиционные системы управления контентом, такие как WordPress, Joomla и Drupal, предоставляют широкий набор возможностей, включая работу с базами данных, плагины и административные панели. Однако для создания веб-портфолио подобные решения являются избыточными. Они требуют постоянных обновлений, настройки безопасности и снижают уровень контроля

над кодом, что противоречит задачам демонстрации профессиональных навыков программиста.

Также в рамках анализа рассматривались конструкторы сайтов без программирования, такие как Tilda и Wix. Несмотря на простоту использования, данные платформы не позволяют продемонстрировать владение языками программирования, архитектурой веб-проекта и инструментами разработки. Использование визуальных конструкторов для создания портфолио программиста снижает профессиональное восприятие ресурса и не соответствует целям технической самопрезентации. [7]

Наиболее рациональным инструментом для создания веб-портфолио являются статические генераторы сайтов. Они формируют веб-ресурс в виде набора готовых HTML-файлов, что обеспечивает высокую скорость загрузки страниц, повышенный уровень безопасности и простоту размещения на хостинге. Отсутствие серверной логики и баз данных существенно упрощает сопровождение проекта и снижает вероятность возникновения уязвимостей. [9]

Среди статических генераторов особое место занимает Lektor CMS — гибкая и мощная система управления статическим контентом с открытым исходным кодом. Lektor является кроссплатформенным решением и работает в операционных системах Linux, macOS и Windows. Система реализована на языках программирования Python и Node.js, что обеспечивает её расширяемость и возможность интеграции с другими программными продуктами. Использование Python, который является одним из основных языков программирования, изучаемых в университете, это позволяет эффективно работать с системой и способствует более осознанному и продуктивному процессу разработки. [10]

Lektor CMS был создан как инструмент, сочетающий в себе преимущества классических CMS и статических генераторов сайтов. Он объединяет удобство

управления контентом, характерное для систем вроде WordPress и Drupal, стабильность и безопасность статических генераторов, таких как Jekyll и Hugo, а также гибкость веб-фреймворков. В отличие от большинства CMS, Lektor не функционирует непосредственно на сервере. Разработка и сборка сайта осуществляются локально либо на сервере сборки, после чего готовые статические файлы могут быть размещены на любом веб-хостинге.

Принцип работы Lektor CMS обеспечивает прозрачную файловую структуру проекта, что делает систему особенно удобной для программистов. Контент хранится в виде файлов, легко отслеживается с помощью системы контроля версий Git и может быть синхронизирован с удалёнными репозиториями. Это позволяет эффективно управлять версиями сайта и автоматизировать процесс публикации. [11]

Для размещения готового веб-портфолио в данной работе используется платформа GitHub Pages, предоставляющая бесплатный и стабильный хостинг статических сайтов. Использование GitHub Pages логично дополняет архитектуру проекта на базе Lektor CMS и позволяет одновременно продемонстрировать навыки работы с системой контроля версий и процессами деплоя.

Таблица 1. Сравнение основных инструментов для создания веб-портфолио

Критерий	Конструкторы сайтов	Традиционные CMS	Статические генераторы
Требуется программирование	Нет	Частично	Да
Контроль над кодом	Минимальный	Ограниченный	Полный
Наличие администрирования	Да	Да	Нет
Избыточный функционал	Да	Да	Нет
Безопасность	Средняя	Средняя	Высокая
Подходит для размещения на GitHub Pages	Частично	Нет	Да
Демонстрация навыков программиста	Нет	Условно	Да

Таким образом, анализ современных инструментов разработки позволяет сделать вывод, что Lektor CMS является оптимальным выбором для создания образовательного и профессионального веб-портфолио программиста. Эта система обеспечивает удобство управления контентом, высокий уровень контроля над кодом, безопасность и простоту сопровождения проекта.

В первой главе рассмотрены теоретические основы создания веб-портфолио программиста, его назначение и ключевые функции как инструмента профессиональной самопрезентации. Проанализированы основные виды портфолио и требования к его структуре, что позволило определить оптимальный набор разделов для будущего сайта. Рассмотрены современные технологии разработки, в результате чего установлено, что статические генераторы являются наиболее рациональным решением для персонального портфолио. На основе сравнительного анализа обоснован выбор Lektor CMS и GitHub Pages как инструментов, обеспечивающих удобство разработки, высокую производительность и соответствие поставленным задачам из введения: анализу подходов к созданию портфолио, изучению технологий, выбору инструментов и обоснованию платформы для размещения.

2 Проектирование и реализация веб-портфолио с помощью Lektor CMS

2.1 Проектирование структуры, функциональных требований и дизайн веб-портфолио

На этапе проектирования была определена базовая структура образовательного веб-портфолио, ориентированного на демонстрацию профессионального развития и результатов обучения: главная страница, раздел с учебными работами, раздел об авторе и блок с контактами. Для каждой страницы сформированы функциональные требования: систематизированное представление результатов, наличие ссылок на репозитории и материалы и краткого описания задач проектов и примененных технологий. Схематичное представление структуры веб-портфолио представлено на рисунке 1. В портфолио были включены 46 дисциплин: 3 на первом курсе, 10 на втором курсе, 18 на третьем курсе, 15 на четвертом курсе.

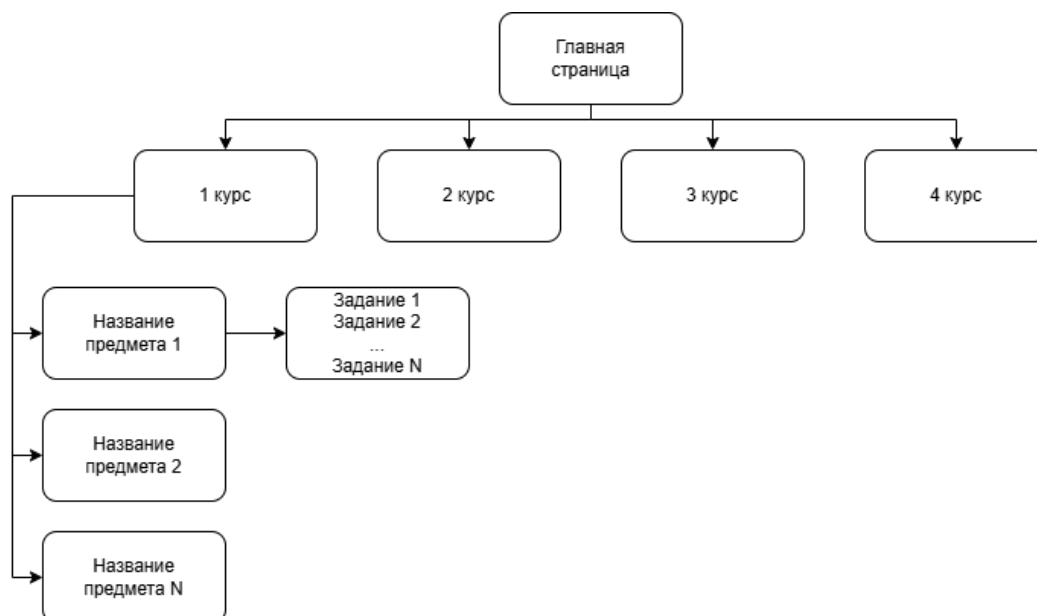


Рисунок 1. Структура образовательного веб-портфолио программиста

Для визуальной части был выбран готовый шаблон для Lektor CMS, наиболее подходящий под формат образовательного портфолио. Он обеспечил минималистичный дизайн, удобную сетку для представления проектов. Шаблон, представленный на рисунке 2, стал основой для дальнейшей настройки структуры и содержимого сайта. [12]



Рисунок 2. Шаблон Resume для Lektor CMS

2.2 Реализация основных разделов и функциональных модулей в Lektor CMS

На этапе практической реализации был создан веб-сайт на основе предварительно спроектированной структуры и выбранного визуального шаблона. Разработка велась в интегрированной среде PyCharm, обеспечивающей удобство работы с кодом и файловой структурой проекта.

Первоначально был установлен исполняемый файл Lektor с официального источника. Для инициализации проекта использована команда `lektor quickstart` в терминале, в процессе выполнения которой были заданы параметры проекта: название (Portfolio), имя пользователя (Ixmak), путь к

проекту, а также принято решение об отказе от создания модуля для ведения блога ввиду его нецелесообразности для данного типа портфолио.

Интеграция выбранного шаблона осуществлена путём создания каталога `themes` в корневой директории проекта и клонирования в него репозитория темы с использованием системы контроля версий Git. Для активации шаблона в конфигурационном файле `Portfolio.lektorproject` добавлена строка `themes = lektor-resume`.

Первичная настройка контента выполнена через редактирование того же конфигурационного файла в разделе `[theme_settings]`. Были заданы основные данные пользователя: фамилия и имя, город проживания, а также контактная информация, включая адрес электронной почты, ссылки на профиль GitHub и аккаунт Telegram. Для персонализации интерфейса заменено стандартное изображение профиля, размещённое по пути `Portfolio/themes/lektor-resume/assets/img`.

Структура навигации сайта была адаптирована под образовательную специфику портфолио. Вместо предусмотренных шаблоном разделов «Опыт работы», «Навыки» и «Проекты» созданы разделы, соответствующие годам обучения: «1 курс», «2 курс», «3 курс» и «4 курс». Данная модификация выполнена через редактирование файла шаблона `layout.html`, расположенного в директории `Portfolio/themes/lektor-resume/templates`. (см. рисунок 3) Для отображения контента в созданных разделах использован стандартный шаблон страницы `page.html`.

```

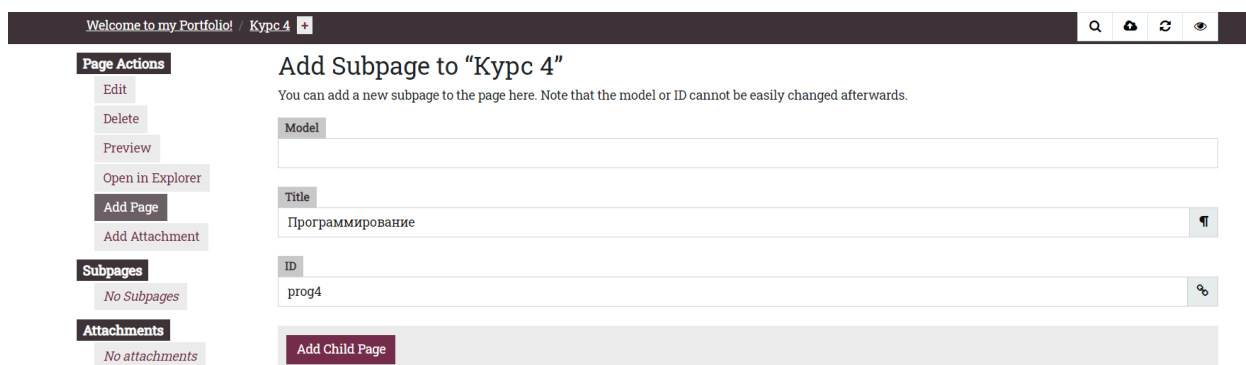
58 <div class="collapse navbar-collapse" id="navbarSupportedContent">
59   <ul class="nav navbar-nav">
60     <li {% if this._path == '/' %} class="nav-item active" {% else %} class="nav-item" {% endif %}><a class="nav-link"
61       {% for href, title in [
62         ['/kurs-1', 'Курс 1'],
63         ['/kurs-2', 'Курс 2'],
64         ['/kurs-3', 'Курс 3'],
65         ['/kurs-4', 'Курс 4']
66       ] %}
67     <li {% if this.is_child_of(href) %} class="nav-item active" {% else %} class="nav-item" {% endif
68       %}><a class="nav-link" href="{{ href|url }}">{{ title }}</a></li>
69     {% endfor %}
70   </ul>
71 </div>

```

Рисунок 3. Фрагмент файла layout.html

Для организации содержимого внутри каждого курса разработана модель дочерних страниц, отвечающих за отдельные учебные дисциплины. Создан новый шаблон mini-page.html, наследуемый от page.html. В каталоге Portfolio/themes/lektor-resume/models созданы соответствующие файлы моделей: mini-page.ini и модифицирован page.ini (см. Приложение Б). Данное решение позволяет тиражировать структуру страниц дисциплин с единообразными полями.

Наполнение сайта контентом выполнялось через встроенную административную панель Lektor, доступную по адресу локального сервера (lektor server). Для каждой дисциплины создавалась дочерняя страница с уникальным идентификатором, заголовком и контентом в соответствии с заданной моделью. (см. рисунок 4)



The screenshot shows the Lektor admin interface. At the top, there's a header with "Welcome to my Portfolio!" and "Курс 4". Below the header, there's a sidebar with "Page Actions" (Edit, Delete, Preview, Open in Explorer, Add Page, Add Attachment) and "Subpages" (No Subpages). The main area is titled "Add Subpage to "Курс 4"" and contains a form with fields for "Model", "Title" (filled with "Программирование"), and "ID" (filled with "prog4"). There's also an "Add Child Page" button.

Рисунок 4. Создание дочерней страницы

Для обеспечения удобной навигации со страницы курса к страницам дисциплин в тело (body) раздела курса добавлены гиперссылки на соответствующие дочерние страницы.

Edit "Курс 4"

Title	Курс 4
Body	Здесь работы за 4 курс. <pre><div class="disciplines-columns"> Программирование </div></pre>
Subpages	

Рисунок 5. Редактирование страницы курса

В целях улучшения визуального восприятия и юзабилити, список дисциплин на странице курса был оформлен не простым перечислением, а в виде сетки из трёх колонок, где каждый элемент представлен в стилизованном виде кнопки. Данный эффект достигнут за счёт добавления пользовательских CSS-правил в файл `Portfolio/themes/lektor-resume/assets/css/resume.min.css` (код представлен в Приложении В).

Контент каждой страницы дисциплины структурирован по единому принципу: заголовок с названием предмета, вводный текст с указанием номера курса, далее — перечень лабораторных или практических работ. Для каждой работы приводится краткое описание задания, использованные технологии и активные ссылки на отчёт и сопутствующие материалы в репозитории GitHub. Для дисциплин, изучаемых в течение двух семестров, контент логически разделён по семестрам.

2.3 Развертывание и публикация веб-портфолио на GitHub Pages

Завершающим этапом работы стала публикация готового веб-портфолио в открытом доступе.

Процесс публикации был организован следующим образом:

1. В аккаунте на GitHub создан новый репозиторий для размещения файлов проекта.
2. Сгенерирован персональный токен доступа (Personal Access Token), необходимый для безопасной аутентификации при автоматизированной загрузке файлов.
3. В корневом конфигурационном файле проекта `Portfolio.lektorproject` добавлена секция `[servers.ghpages]`. В рамках этой секции заданы имя сервера публикации и целевой адрес (`target`) в формате `ghpages+https://<username>:<token>/<repository-name>`, где `<username>` — логин пользователя GitHub, `<token>` — сгенерированный ключ доступа, `<repository-name>` — название созданного репозитория.
4. Для выполнения финальной сборки статических файлов и их отправки на GitHub использована команда `lektor deploy ghpages`. В результате выполнения данной команды система Lektor осуществила сборку финальной версии сайта и загрузила все необходимые файлы в указанный репозиторий, активировав механизм GitHub Pages.

После завершения деплоя веб-портфолио стало доступно по постоянному публичному URL (см. приложение Г), предоставляемому сервисом GitHub Pages. Таким образом, достигнута ключевая практическая цель работы — создание функционирующего, наполненного контентом и опубликованного в сети веб-ресурса, демонстрирующего образовательные и профессиональные достижения.

Во второй главе курсовой работы был реализован практический этап разработки веб-портфолио программиста с использованием системы управления статическим контентом Lektor CMS. В ходе выполнения работы были спроектированы структура и функциональные требования веб-ресурса,

выполнена адаптация визуального шаблона под образовательную специфику портфолио и реализовано наполнение сайта контентом на основе разработанных моделей страниц.

В результате практической реализации выполнена настройка проекта, обеспечена логичная навигация между разделами, унифицировано представление учебных дисциплин и реализована публикация готового веб-портфолио на платформе GitHub Pages. Полученный результат подтверждает возможность эффективного использования Lektor CMS для создания и развертывания статических веб-ресурсов, соответствующих задачам профессиональной и образовательной самопрезентации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель — разработано и опубликовано веб-портфолио программиста с использованием системы управления статическим контентом Lektor CMS. В рамках работы были последовательно решены все поставленные задачи, сформулированные во введении.

В рамках работы был проведен анализ современных подходов к созданию профессиональных веб-портфолио программиста. Рассмотрены назначение портфолио, его основные виды и требования к структуре, а также выявлены ключевые разделы, необходимые для эффективной профессиональной самопрезентации. Были изучены технические особенности и преимущества статического генератора Lektor CMS, выполнено сравнение с альтернативными инструментами веб-разработки и обоснован выбор Lektor CMS как оптимального средства для создания веб-портфолио программиста.

Также в процессе была спроектирована структура и дизайн веб-портфолио, определены функциональные требования к его разделам и выбрано визуальное оформление, соответствующее образовательной специфике ресурса. Одна из задач была реализована на практическом этапе работы и включала настройку проекта в среде Lektor CMS, разработку пользовательских моделей страниц, адаптацию шаблона и наполнение сайта контентом. Были настроены хостинг и процесс публикации веб-портфолио на платформе GitHub Pages, что позволило разместить готовый статический сайт в открытом доступе.

Таким образом, в результате выполнения курсовой работы был создан функционирующий веб-портфолио, соответствующий современным требованиям профессиональной самопрезентации в сфере информационных технологий и демонстрирующий практические навыки работы с Lektor CMS, системой контроля версий Git и инструментами деплоя.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. Исследовательская работа “Создание web-портфолио” // Инфоурок. URL: <https://infourok.ru/issledovatel'skaya-rabota-sozdanie-ebportfolio-957722.html> (дата обращения: 19.11.2025).
2. Personal Portfolio Creation Guide // Theseus. URL: <https://www.theseus.fi/handle/10024/890143> (дата обращения: 19.11.2025).
3. Al-Hmouz, R., Al-Kabi, M., & Alsmadi, I. Factors affecting learning performance in web-based learning environment: A systematic review // Education and Information Technologies, 2023. URL: <https://link.springer.com/article/10.1007/s10639-023-12090-z> (дата обращения: 20.11.2025).
4. Создание веб-портфолио: пошаговая инструкция // Dzen. URL: <https://dzen.ru/a/Ybx0yUtGwBynn2jZ> (дата обращения: 20.11.2025).
5. Website Design // Colorlib URL: <https://colorlib.com/wp/developer-portfolios/> (дата обращения: 21.12.2025).
6. 10 examples of a good developer portfolio // Kerthin URL: <https://dev.to/kerthin/10-examples-of-a-good-developer-portfolio-for-your-inspiration-2f88> (дата обращения: 21.12.2025).
7. Как создать сайт-портфолио: пошаговая инструкция // SiteRost. URL: <https://siterost.net/post/kak-sozdat-sajt-portfolio-poshagovaya-instruktsiya> (дата обращения: 20.11.2025).
8. Как делать верстку портфолио // SSL-Team. URL: https://ssl-team.com/blog/kak-delat-verstku-portfolio/?utm_referrer=https%3A%2F%2Fyandex.ru%2F (дата обращения: 22.11.2025).
9. Best Static Site Generators // TechRadar. URL: <https://www.techradar.com/best/static-site-generators> (дата обращения: 22.11.2025).
10. Python CMS: Lektor // Quirntagroup. URL: <https://quintagroup.com/cms/python/lector> (дата обращения: 01.12.2025).
11. Lektor CMS Guide // FullStackPython. URL: <https://www.fullstackpython.com/lector.html> (дата обращения: 28.11.2025).
12. Lektor Theme Resume // GitHub. URL: <https://github.com/hannylicious/lector-theme-resume/tree/e2c32d5a4ab877fa6029a0b2e3edb13d6425afab> (дата обращения: 28.11.2025).

13.Documentation // Lektor Static Content Management System URL:
<https://www.getlektor.com/docs/> (дата обращения: 21.12.2025).

ПРИЛОЖЕНИЕ А

Ссылки на существующие веб-портфолио.

1. <https://www.alexnaraghi.com/>



2. <https://tamalsen.dev/>



3. <https://www.lars-olson.com/>



4. <https://bepatrickdavid.com/>



ПРИЛОЖЕНИЕ Б

Файлы моделей для page и mini-page

page.ini

```
[model]
name = Page
label = {{ this.title }}

[fields.title]
label = Title
type = string

[fields.body]
label = Body
type = markdown

[fields.children]
label = Subpages
type = markdown
item_type = mini-page

[fields.nav_bar]
label = Nav Bar Item
type = boolean
checkbox_label = If true, then the page title will create a link in the Major Nav bar.
default = false

[fields.meta_description]
label = Page Description (160 chars)
type = string

[fields.meta_keywords]
label = Page Keywords
type = string
```

mini-page.ini

```
name = Mini Page
label = {{ this.title }}

[fields.title]
label = Title
type = string

[fields.body]
label = Body
type = markdown

[fields.meta_description]
label = Page Description (160 chars)
type = string

[fields.meta_keywords]
label = Page Keywords
type = string
```

ПРИЛОЖЕНИЕ В

Фрагмент файла resume.min.css

```
.course-tag {
  display: inline-block;
  margin: 6px 6px 0 0;
  padding: 15px 16px;
  background-color: #636f7a;
  color: #000000;
  border-radius: 20px;
  font-size: 1 rem;
  font-weight: 500;
  text-align: center;
  justify-content: center;
  align-items: center;
}

.disciplines-columns {
  column-count: 3;
  column-gap: 3rem;
  margin: 2rem 0;
}

.disciplines-columns .course-tag {
  display: inline-block;
  width: 100%;
  margin-bottom: 0.8rem;
  break-inside: avoid;
}

@media (max-width: 768px) {
  .disciplines-columns {
    column-count: 1;
  }
}
```

ПРИЛОЖЕНИЕ Г

Ссылка на веб-портфолио

<https://nasirdn.github.io/>

